The Zynq Book Tutorial 4

IP Creation

v1.2, May 2014

Revision History

Date	Version	Changes
22/10/2013	1.0	First release for Vivado Design Suite version 2013.3
28/01/2014	1.1	Updated for changes in Vivado Design Suite version 2013.4
06/05/2014	1.2	Updated for changes in Vivado Design Suite version 2014.1

Introduction

The exercises in this tutorial will guide you through the process of creating custom IP modules, that are compatible with Vivado IP Integrator, from a variety of different sources. All created IP will be compatible with the Xilinx supported AXI-Lite interface, and will be connected as slave devices when implemented in Vivado IP Integrator.

All IP creation methods that are covered here coincide with those covered in the book:

- HDL
- MathWorks HDL Coder
- Xilinx Vivado HLS

The tutorial is split into three exercises, and is organised as follows:

Exercise 4A - In this exercise, HDL will be used to create a controller which will allow the LEDs on the ZedBoard to be controlled by software running on the PS. The Create and Package IP Wizard will be used to create an AXI-Lite interface wrapper which the LED control process and interface will be added to. The IP packaging process will then be used to create an IP block which is compatible with IP Integrator.

Exercise 4B - HDL Coder, the MathWorks HDL generation tool, will be explored in this exercise. A Least Mean Squares (LMS) adaptive filter will be created and tested in the Simulink workspace. The LMS design will then be used to generate HDL code by invoking the HDL Coder Workflow Advisor, where the option to generate a Xilinx IP Core will be selected. The various stages of the workflow will verify the design to ensure that it is HDL Coder compliant and produce the HDL code in a format that is compatible with IP Integrator.

Exercise 4C - In this final exercise, Vivado HLS will be used to create an IP core for a Numerically Controlled Oscillator (NCO). An existing C-code algorithm will be simulated for testing, and ran through the various stages of synthesis in order to create an IP Integrator compatible IP core.

Exercise 4A Creating IP in HDL

With Zynq devices comprising of both PS and PL parts, most IP that is created to run in PL should be able to communicate with software running on the PS. This requires that IP should be packaged with an interface that is compatible with the PS (in this case the AXI interface).

When creating IP in HDL, Vivado provides a set of AXI interface templates which can be created and customised via the *Create and Package IP Wizard*. The wizard, as the name suggests, facilitates two major functions: the creation of AXI4 IP peripherals; and the packaging of existing source files into an IP package which is compatible with the IP Integrator tool.

In this exercise we will actually be making use of both of these features to firstly create an AXI4-Lite IP template to which we will add functionality to allow the LEDs on the ZedBoard to be controlled via a software application running on the Zynq PS. Once the functionality has been added to the template, the source files will be packaged into an IP Integrator compatible IP block which will be included in a simple Zynq processor system.

We will start by creating a new Vivado project.

(a) Launch Vivado by double-clicking on the Vivado desktop icon: , or by navigating to **Start > All Programs > Xilinx Design Tools > Vivado 2014.1 > Vivado 2014.1**

(b) Select ***Create New Project*** from the *Getting Started* screen.

(c) The *New Project* dialogue will open. Click ***Next***.

(d) At the Project Name dialogue, enter ***led_controller*** as the ***Project name*** and ***C:/Zynq_Book*** as ***Project location***.

Make sure that you select the option to ***Create project subdirectory***. Ensure that all options match Figure 4.1.

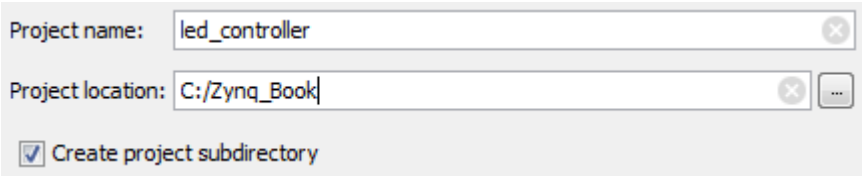
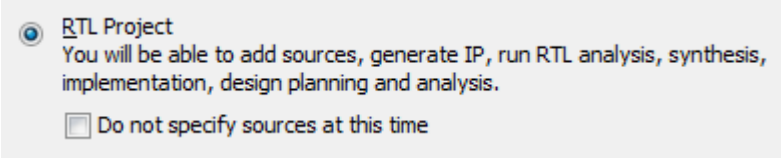


Figure 4.1: Vivado Project Name specification - led_controller

Click ***Next***.

(e) Select ***RTL Project*** at the *Project Type* dialogue, and ensure that the option ***Do not specify sources at this time*** is not selected:



Click ***Next***.

(f) Select ***VHDL*** as the ***Target language*** in the *Add Sources* dialogue.

If existing sources, in the form of HDL or netlist files, were to be added to the project they could be imported at this stage.

As we do not have any sources to add to the project, click ***Next***.

(g) The *Add Existing IP (optional)* dialogue will open.

If existing IP sources were to be included in the project, they could be added here.

As we do not have any existing IP to add, click ***Next***.

(h) The *Add Constraints (optional)* dialogue will open.

This is the stage were any physical or timing constraints files could be added to the project.

As we do not have any constraints files to add, click ***Next***.

(i) The *Default Part* dialog will open. Here we will be selecting the Zynq part which we are targeting. In this particular case we will be targeting the Zynq-7020 on the ZedBoard, but if you have a different development board, it is easy to choose your particular board instead.

Select ***Boards*** from the *Specify* pane, ***ZedBoard Zynq Evaluation and Development Kit*** as the *Display Name*, and finally select the *Board Rev* which you have. In Figure 4.2 **version C** of the **ZedBoard** has been selected.

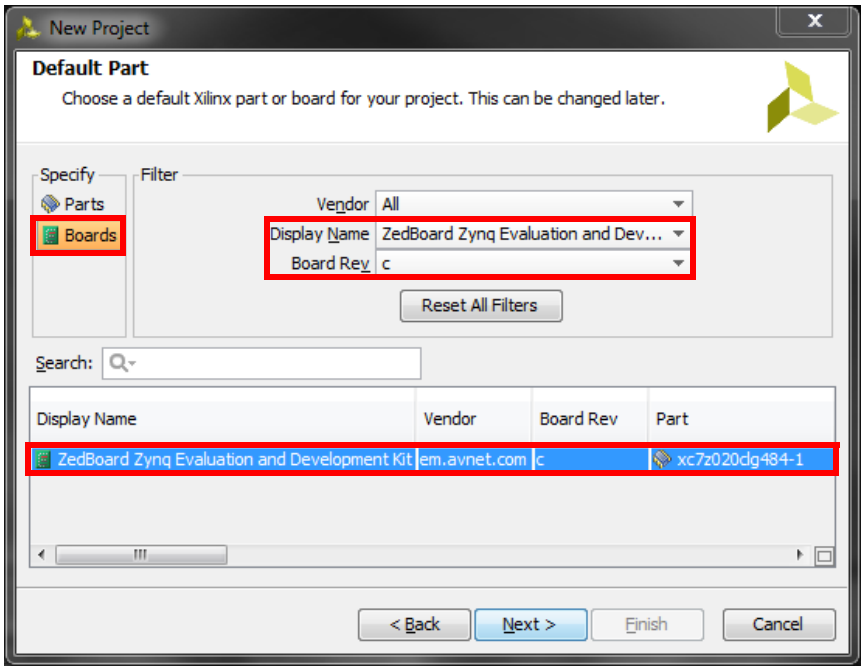


Figure 4.2: Vivado Default Part dialogue

Click ***Next***.

(j) Review the *New Project Summary* dialogue, and click ***Finish*** to create the project.

With the new project created, we can begin the process of creating our HDL-based IP.

(k) From the menu bar, select **Tools > Create and Package IP ...**, as in Figure 4.3, to launch the ***Create and Package IP Wizard***.

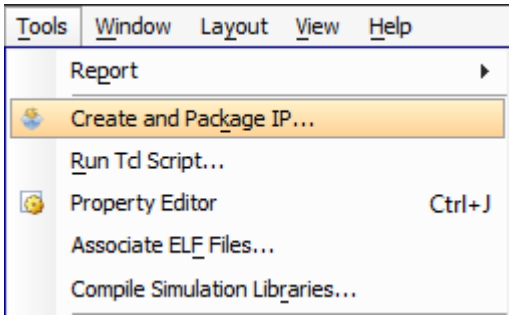


Figure 4.3: Create and Package IP menu bar selection

(l) The *Create and Package IP Wizard* dialogue will launch, as shown in Figure 4.4.

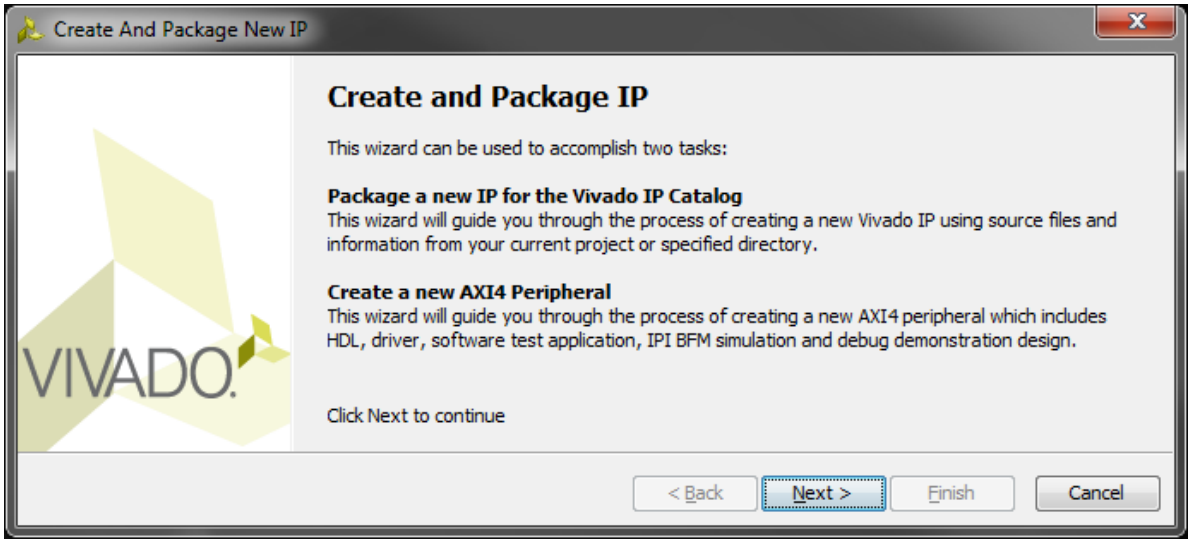


Figure 4.4: Create and Package IP Wizard dialogue

Click ***Next***.

The *Choose Create Peripheral or Package IP* dialogue (Figure 4.5) is where we specify whether to create a new peripheral template file or to package existing source files into an IP core.
In our case we want to create a new IP template.

(m) Select ***Create new AXI peripheral***, as shown in Figure 4.5.

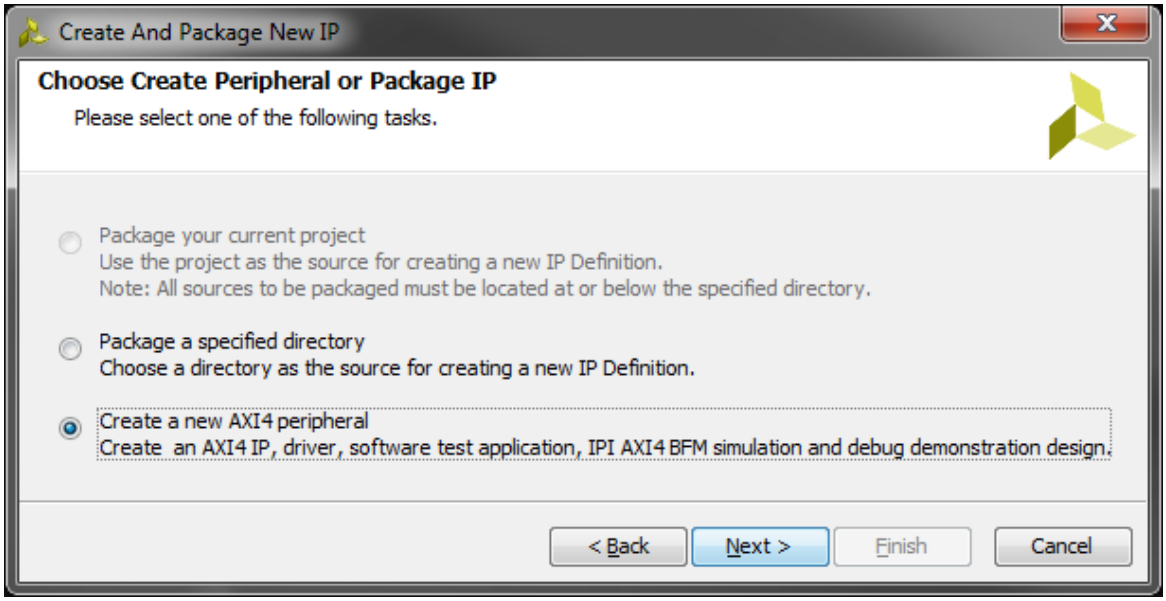


Figure 4.5: Choose Create or Package IP dialogue

Click ***Next***.

The *Peripheral Details* dialogue allows you to specify the **Vendor, Library, Name and Version (VLNV)** information, as well as other details, for the new peripheral, leaving the *IP Location* as the default.

(n) Fill in the details as shown in Figure 4.6.

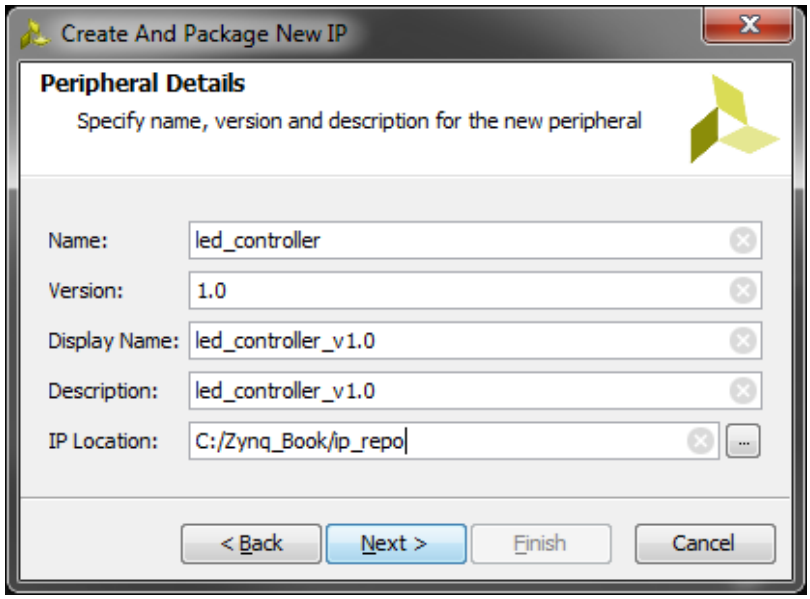


Figure 4.6: Peripheral Details dialogue

Click ***Next***.

The *Add Interface* dialogue allows you to specify the AXI4 interface(s) that will be present in your custom peripheral. Here you can specify:

- Number of interfaces
- Interface type (AXI-Lite, AXI-Stream or AXI-Full)
- Interface mode (slave or master)
- Interface data width

Features specific to individual interface types will also be available when the corresponding type is selected.

As our peripheral is a simple controller for the LEDs which only requires single values to be transferred to it, an AXI-Lite slave interface is sufficient. Only one memory mapped register is required for our simple controller, but as the minimum number that can be specified in the dialogue is 4, we will choose that.

(o) Specify the *Add Interface* dialogue as shown in Figure 4.7.

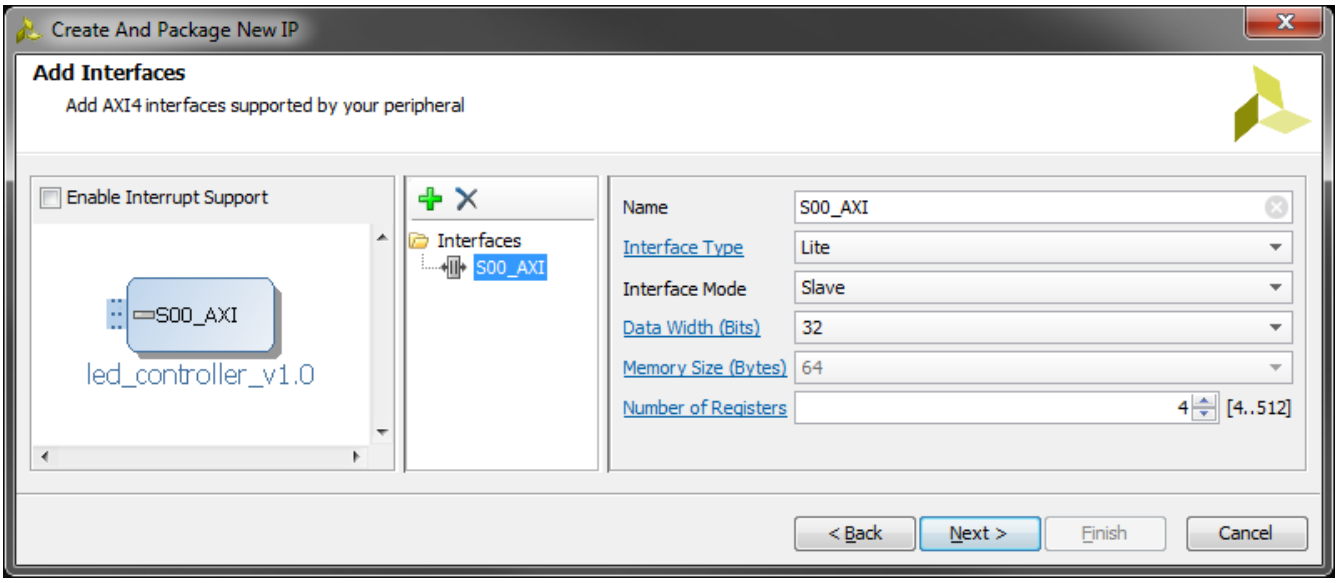


Figure 4.7: Add Interface dialogue

Click ***Next***.

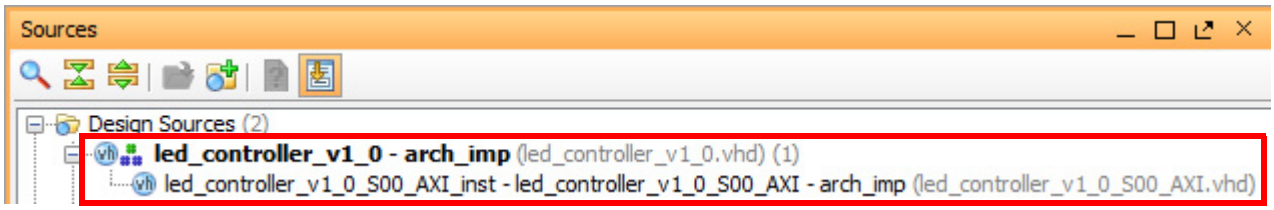
(p) Review the information in the *Create Peripheral* dialogue, which details the output files which will be created.

Select the option to ***Edit IP***. This will create the IP peripheral files and create a new Vivado project where the functionality of the peripheral can be modified in the source HDL code, and then packaged.

Click ***Finish*** to close the *Wizard* and create the peripheral template.

A new Vivado project, named **edit_led_controller_v1_0**, will open.

In the *Sources* pane, you should see two HDL source files:

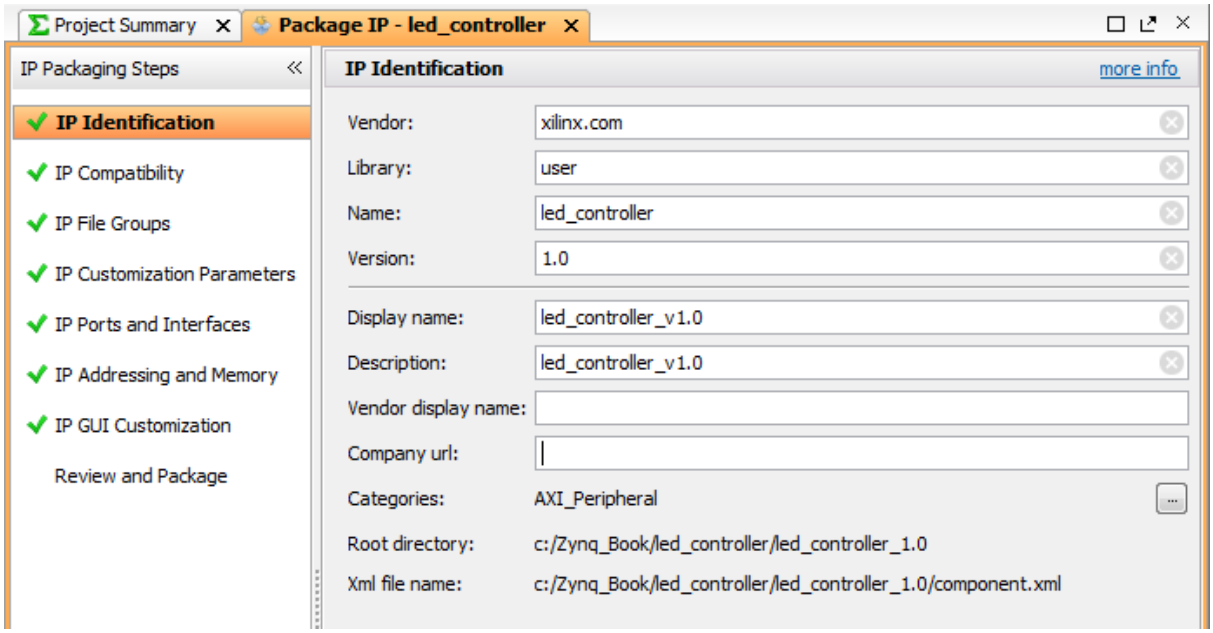


As we specified our target language as **VHDL** in Step (f) earlier, the template files have been generated in VHDL. Had we specified Verilog as the target language, Verilog source files would have been created.

The two source files are:

- **led_controller_v1_0.vhd** — This file instantiates all AXI-Lite interfaces. In this case, only one interface is present.
- **led_controller_v1_0_S00_AXI.vhd** — This file contains the AXI4-Lite interface functionality which handles the interactions between the peripheral in the PL and the software running on the PS.

The *IP Packager* pane will also be open in the *Workspace*:



The information that we specified about our peripheral in Step (n) will be visible.

We can now add the functionality to our **led_controller** peripheral. We will be adding a new output port to the peripheral template to allow it to connect to the LED pins on the Zynq device, as well as assigning the value received from the Zynq PS to the new output port.

(q) Open ***led_controller_v1_0_S00_AXI.vhd*** by double-clicking on it in the *Sources* pane. The file will open in the *Workspace*.

(r) Scroll down until you see the following comment in the entity port declaration:

```
-- Users to add ports here
```

and add the following port definition directly below the comment:

```
LEDs_out : out std_logic_vector(7 downto 0);
```

This creates a new output port with a width of 8-bits (a single bit to represent each of the LEDs on the ZedBoard).

(s) Scroll to the bottom of the file. You should see the following comment:

```
-- Add user logic here
```

and add the following port/signal assignment:

```
LEDs_out <= slv_reg0(7 downto 0);
```

This assigns the value that is received from the Zynq PS (stored in the signal slv_reg0) to the output port that we created in the previous step.

(t) Save the file by selecting **File > Save File** from the Menu Bar, or using the keyboard shortcut **Ctrl+S**.

(u) Open ***led_controller_v1_0.vhd*** by double-clicking on it in the *Sources* pane. The file will open in the *Workspace*.

We must once again create a new output port to the top-level source file, and map it to the equivalent port that we created in the AXI4-Lite interface file in the previous steps.

(v) Scroll down until you see the following comment in the entity port declaration:

```
-- Users to add ports here
```

and add the following port definition directly below the comment:

```
LEDs_out : out std_logic_vector(7 downto 0);
```

As we added a new port to the AXI4-Lite interface file, we must also add it to the component declaration in the top-level file.

(w) Scroll down until you see the comment:

```
-- component declaration
```

A few lines further down you will see the component port declaration:

```
port (
```

Inside the port declaration (below the "`port (`"line), add the following output port definition:

```
LEDs_out : out std_logic_vector(7 downto 0);
```

Finally, we must add a port mapping between the LED output ports of the top-level file and the AXI4-Lite interface file.

(x) Scroll down until you see the comment:

```
-- Instantiation of Axi Bus Interface S00_AXI
```

A few lines further down you will see the component port map:

```
port map (
```

Inside the component port map (below "`port map (`" line), add the following port map:

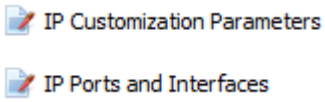
```
LEDs_out => LEDs_out,
```

(y) Save the file.

Now that we have made the necessary modifications to the peripheral source files, we must repackage the IP to merge the changes.

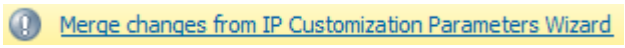
(z) Return to **IP Packager** by selecting the ***Package IP - led_controller*** tab in the *Workspace*:

IP Packager will detect the changes to the source files, and the areas which need refreshed will be highlighted with the following icon: . You should see that the following two areas need refreshed:



(aa) Select ***IP Customization Parameters*** in the *IP Packager* pane.

You should see the following information message at the top of the pane:



Click ***Merge changes from IP Customization Parameters Wizard***

This will update the IP Packager information to reflect the changes made in the HDL source files.

NOTE: This process updates IP Packager information for all areas. You should see that the area of IP Ports no longer needs updated, and the icon has now been removed.

To verify that IP Packager has updated the IP Ports area, we will open it and check.

(ab) Select ***IP Ports and Interfaces*** from the *IP Packager* pane.

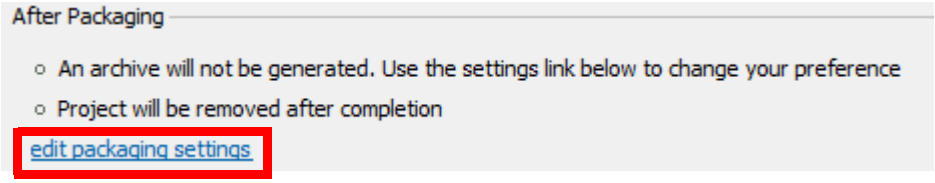
You should notice that the **LEDs_out** port that we added to the source files has been added to the *IP Ports and Interfaces* pane:

Name	Direction	Driver Value	Size Left	Size Left Dependency	Size Right	Size Right Dependency	Type Name
LEDs_out	out		7		0		std_logic_vector

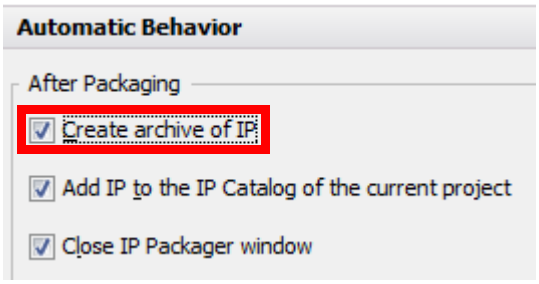
The final step in creating our new IP peripheral, is to package the IP.

(ac) Select ***Review and Package*** from the *IP Packager* pane.

(ad) In the *After Packaging* panel, click ***edit packaging settings*** at the bottom:



(ae) In the *Automatic Behaviour* panel, enable the option to ***Create archive of IP***:



This makes a ZIP file archive of the packaged IP.

(af) Click ***OK*** to apply the setting.

(ag) Review the information provided in the *Review and Package* window, and click ***Re-Package IP***.

(ah) The changes made to the IP peripheral will be included in the repackaged IP, and the Vivado project will close.

We will now return to our original Vivado project, and create a simple Zynq processor block design to check that the functionality of our LED controller peripheral.

To start, we will create a new Block Design and add the IP peripheral which we just created to the design.

(ai) In the *Flow Navigator* window, select ***Create Block Design*** from the *IP Integrator* section.

Enter ***led_test_system*** in the *Design name* box, and click ***OK*** to create the blank design.

(aj) Right-click anywhere in the blank canvas, and select ***Add IP***. Alternatively, use the keyboard shortcut ***Ctrl+I***. This will bring up to pop-up IP Catalog window.
Enter ***led*** in the *Search* box, and double-click ***led_controller_v1_0*** to add an instance of the LED controller IP to the design.

An led_controller_v1_0 block will now be present in the block design, as shown in Figure 4.8.

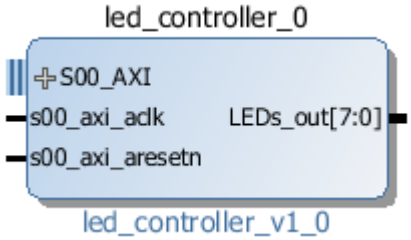


Figure 4.8: led_controller block

The 8-bit **LEDs_out** port that we added to the peripheral is present on the right side of the block.

To enable the peripheral to connect to the LEDs on the ZedBoard, we must make the **LEDs_out** port external. This allows the output port to be connected to specific physical pins on the Zynq device, which are connected to the LEDs.

(ak) Hover the mouse pointer over the **LEDs_out** interface (the little black stub next to the interface name) on the **led_controller** block until the cursor changes to a pencil. Right-click and select ***Make External***. Alternatively, select the interface and use the keyboard shortcut ***Ctrl+T***.

The block design should now resemble Figure 4.9.

Figure 4.9: led_controller block with external port

The next step is to add a Zynq Processing System block to the design and connect the LED Controller to it.

(al) Add an instance of the **Zynq7 Processing System**, using the same procedure as in Step (aj).

(am)The *Designer Assistance* message at the top of the canvas will appear:

Click ***Run Block Automation*** and select ***processing_system7_0***.

An information message will appear. Ensure that ***Apply Board Preset*** is selected, and click ***OK***.

This will make all necessary modifications to the Zynq processing system that relate to the board preset (in this case the ZedBoard) and make required external connections.

The next step that has to be carried out to the block design, is to connect the LED Controller to the Zynq Processing System. This step can also be carried out using Designer Assistance.

(an) In the *Designer Assistance* message, click ***Run Connection Automation*** and select ***led_controller_0/S00_AXI***.

An information message will appear. Click ***OK***.

This will add some additional blocks to the design which are required to connect the LED Controller to the Zynq Processing System.

Our block design is now complete.

(ao) Validate the design by selecting **Tools > Validate Design** from the *Menu Bar*. Alternatively, select the ***Validate Design*** button,  , from the *Main Toolbar*, or use the keyboard shortcut ***F6***.

Dismiss the *Validate Design* message by clicking ***OK***.

We can now generate the HDL files for the design.

(ap) In the *Sources* pane, right-click on the **led_test_system** block design and select ***Create HDL Wrapper***.

Select ***Let Vivado manage wrapper and auto-update*** and click ***OK***.

This will create the top-level HDL file for the design.

We must now connect the LEDs_out port of the design to the correct pins on the Zynq device. This is done through the specification of constraints in an XDC file.

(aq) In the *Flow Navigator* window, select ***Add Sources*** from the *Project Manager* section.

The *Add Sources* dialogue will open.

Select ***Add or Create Constraints***, and click ***Next***.

(ar) Click ***Create File...***

The *Create Constraints File* dialogue will open.

Select ***XDC*** as the *File type* and enter ***led_constraints*** as the *File name*.

Click ***OK***.

(as) Click ***Finish*** to create the file and close the dialogue.

(at) In the ***Sources*** tab, expand the **Constraints** entry and open the newly created XDC file by double-clicking on ***led_constraints.xdc***.

The file will open in the *Workspace*.

(au) Add the following lines to the constraints file. Alternatively, they can be copied from the source file available at **C:\Zynq_Book\sources\led_controller**:

```
set_property PACKAGE_PIN T22        [get_ports {LEDs_out[0]}]
set_property IOSTANDARD LVCMOS33    [get_ports {LEDs_out[0]}]
set_property PACKAGE_PIN T21        [get_ports {LEDs_out[1]}]
set_property IOSTANDARD LVCMOS33    [get_ports {LEDs_out[1]}]
set_property PACKAGE_PIN U22        [get_ports {LEDs_out[2]}]
set_property IOSTANDARD LVCMOS33    [get_ports {LEDs_out[2]}]
set_property PACKAGE_PIN U21        [get_ports {LEDs_out[3]}]
set_property IOSTANDARD LVCMOS33    [get_ports {LEDs_out[3]}]
set_property PACKAGE_PIN V22        [get_ports {LEDs_out[4]}]
set_property IOSTANDARD LVCMOS33    [get_ports {LEDs_out[4]}]
set_property PACKAGE_PIN W22        [get_ports {LEDs_out[5]}]
set_property IOSTANDARD LVCMOS33    [get_ports {LEDs_out[5]}]
set_property PACKAGE_PIN U19        [get_ports {LEDs_out[6]}]
set_property IOSTANDARD LVCMOS33    [get_ports {LEDs_out[6]}]
set_property PACKAGE_PIN U14        [get_ports {LEDs_out[7]}]
set_property IOSTANDARD LVCMOS33    [get_ports {LEDs_out[7]}]
```

This connects each individual bit of the LEDs_out port to a specific pin on the Zynq device. The specific pins are connected to the LEDs on the board.

(av) Save the constraints file.

Our simple design is now complete. We can now generate a bitstream.

(aw) In *Flow Navigator*, select ***Generate Bitstream*** from the *Program and Debug* section.

If a dialogue window appears prompting you to save your design, click ***Save***.

A dialogue window may open requesting that you launch synthesis and implementation

before starting the *Generate Bitstream* process. If it does, click ***Yes*** to accept.

The combination of running the synthesis, implementation and bitstream generation processes back-to-back may take a few minutes, depending on the power of your computer system.

(ax) When bitstream generation is complete a dialogue window will open to inform you that the process as been completed.

Select ***Open Implemented Design***, and click ***OK***.

With the bitstream generation complete, the final step in Vivado is to export the design to the SDK, where we will create the software application that will allow the Zynq PS to control the LEDs on the ZedBoard.

(ay) Select ***File > Export > Export Hardware for SDK...*** from the *Menu Bar*.

The *Export Hardware for SDK* dialogue window will open. Ensure that the options to ***Include bitstream*** and ***Launch SDK*** are selected, and Click ***OK***.

The SDK will launch.

(az) Once the SDK has launched, create a new **Application Project** by selecting **File > New > Application Project** from the *Menu Bar*.

In the *New Project* dialogue, enter ***LED_Controller_test*** as the *Project name*.

By default the option to create a new board support package will be selected.

Click ***Next***.

(ba) In the *Templates* dialogue, select ***Empty Application***, and click ***Finish***.

You should recall that when we created the peripheral in the previous stages of this exercise that a set of software driver files were generated. We must now point SDK to those driver files. This is done by adding a new repository to the SDK project.

(bb)Navigate to **Xilinx Tools > Repositories** in the *Menu Bar*.

In the *Repositories Preferences* window, click on ***New***, as shown in Figure 4.10.

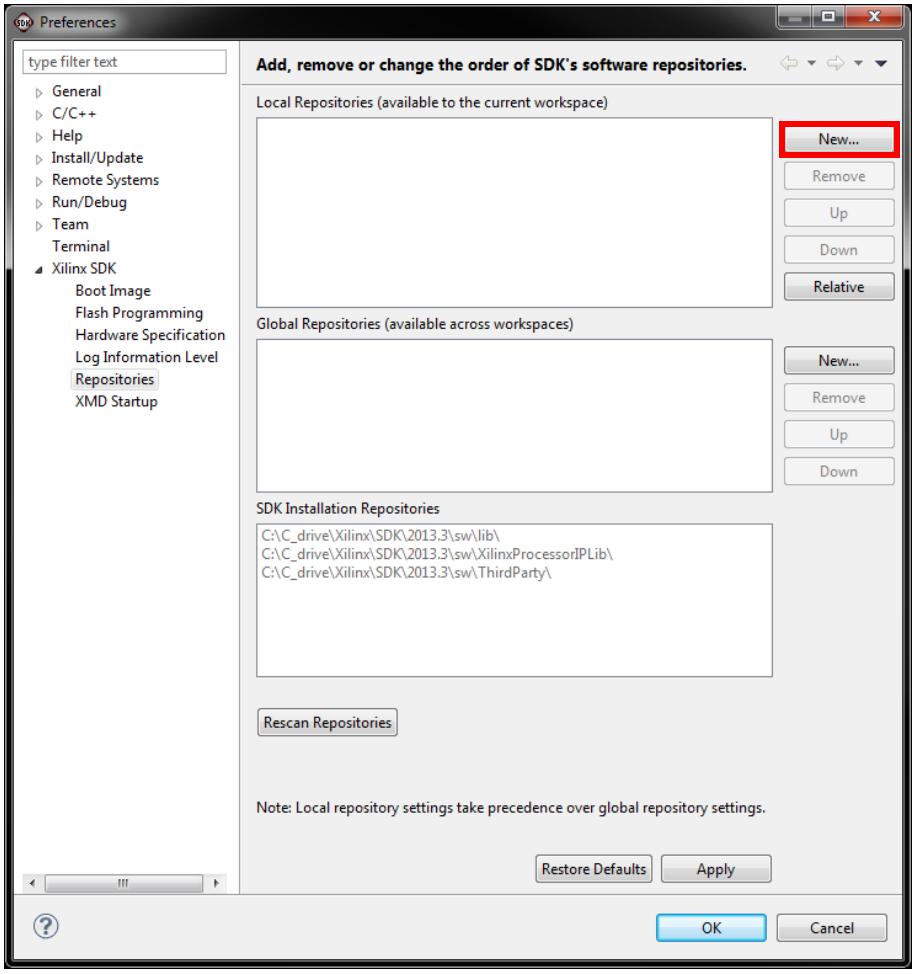


Figure 4.10: SDK Repository Peripherals window

(bc) Browse to the directory **C:\Zynq_Book\ip_repo\led_controller_1.0**, as in Figure 4.11, and click ***OK***.

(bd)Close the Repository Preferences window by clicking ***OK***.

Upon closing the preferences window, SDK will automatically scan the repository and rebuild the project to include the driver files.

We must now assign the newly imported drivers to the LED Controller peripheral.

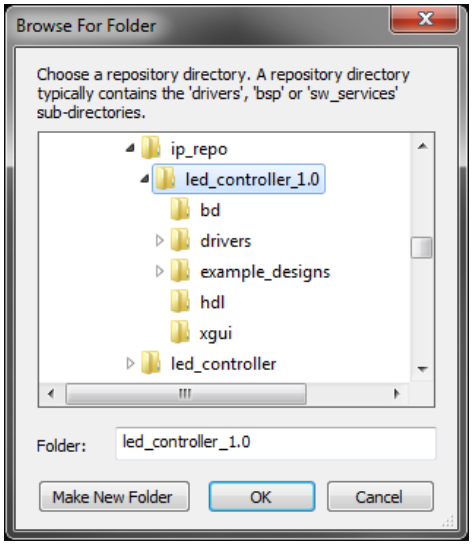


Figure 4.11: led_controller repository selection

(be)The **system.mss** tab should be open in the Workspace. If it is not, open it by expanding **LED_Controller_test_bsp** in *Project Explorer* and double-clicking on ***system.mss.***

(bf) At the top left of the **system.mss** tab, click ***Modify this BSP's Settings***.

The Board Support Package Settings window will open, as in Figure 4.12.

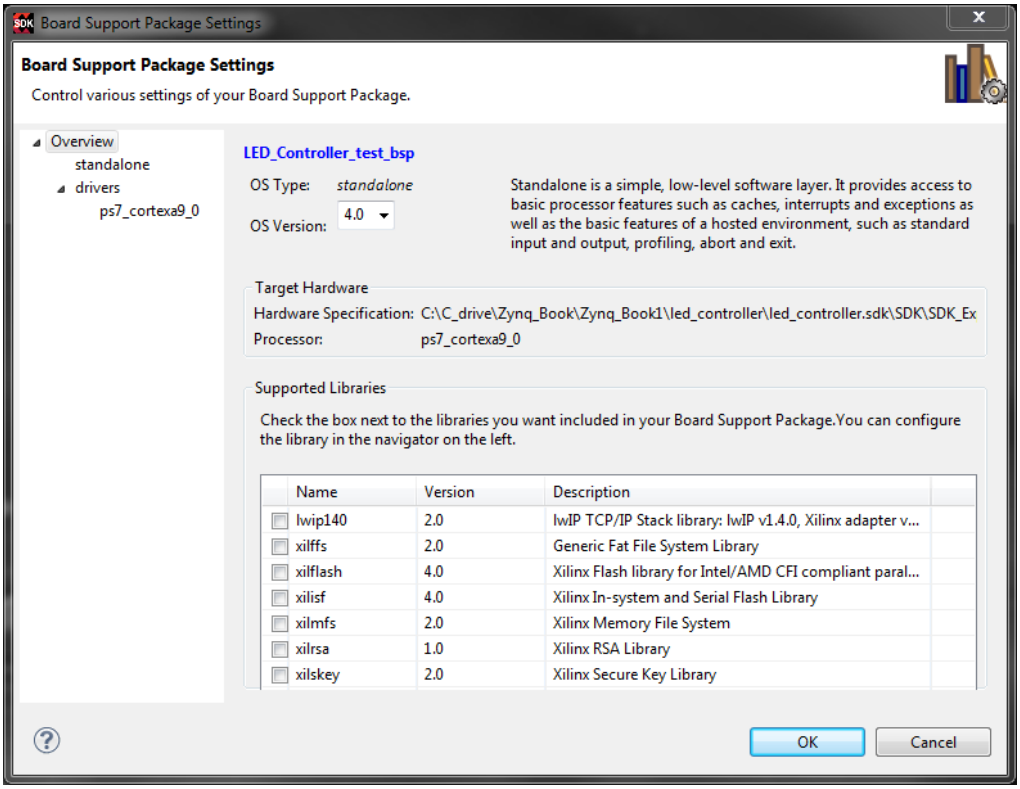


Figure 4.12: Board Support Package Settings window

(bg)Select ***drivers*** from the left-hand menu. From the list of components in the *Drivers* pane, identify **led_controller_0** and select ***led_controller*** from the drop-down menu in the **Driver** column, as shown in Figure 4.13.

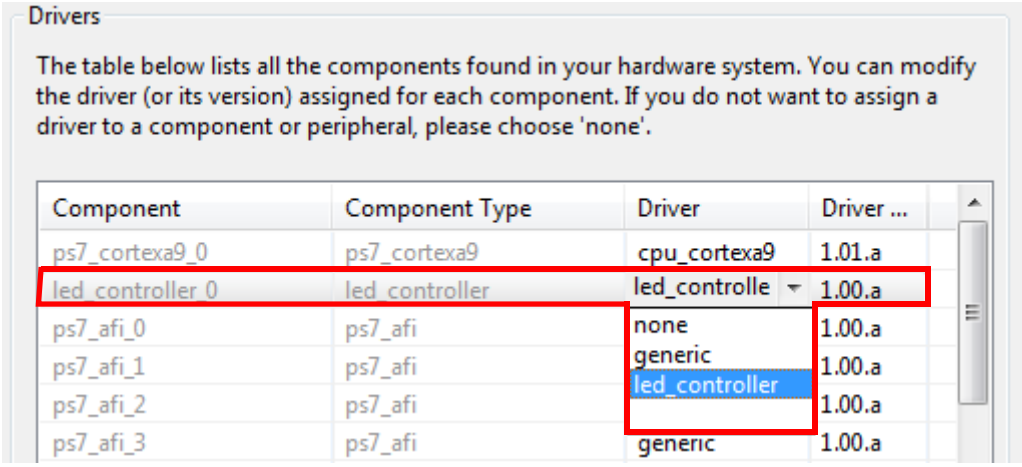


Figure 4.13: LED Controller driver selection

Click ***OK***.

The project will now rebuild.

We can now create a simple C application to control the LEDs. In this instance we will be importing a pre-written source file.

(bh)In *Project Explorer*, right-click on **LED_Controller_Test** and select ***Import***.

In the *Import* window, expand **General** and double-click on ***File System***.

Click ***Browse*** in the top right corner, and navigate to **C:\Zynq_Book\sources\led_controller**.

Click ***OK***.

In the right-hand panel, select ***led_controller_test_tut_4A.c*** and click ***Finish.***

The project will once again rebuild to include the new source file.

Open ***led_controller_test_tut_4A.c*** and examine the functionality.

Before launching the application on the ZedBoard, we must program the Zynq PL and create a new terminal connection.

(bi) From the *Menu Bar*, select **Xilinx Tools > Program FPGA**.

The Bitstream entry should already be populated with the corresponding bitstream that we exported from Vivado earlier.

Click ***Program***, to program the Zynq PL.

NOTE: Once the device has successfully been programmed, the *DONE LED* on the ZedBoard will turn blue.

(bj) Select the ***Terminal*** tab from the *Console* window at the bottom of the workspace, as in Figure 4.14.

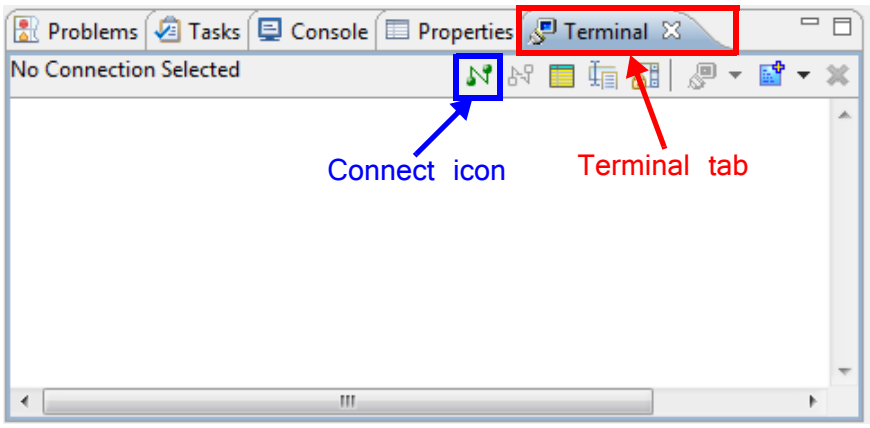


Figure 4.14: SDK Terminal tab

(bk) Click the Connect icon (as highlighted in Figure 4.14).

(bl) The Terminal Settings window will open. Configure the settings as specified in Figure 4.15.

NOTE: The value of the *Port* entry will vary depending on which the USB UART cable is connected to.

In order to determine this value on a Windows system, open the Device Manager and identify the COM port.

(bm) Click OK to initiate the new Terminal connection.

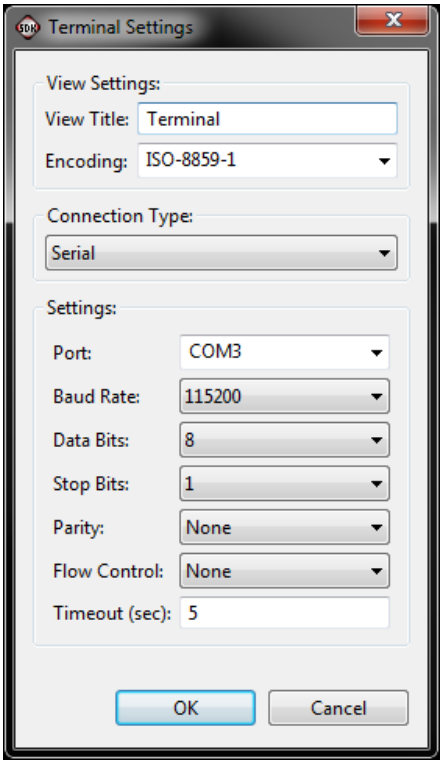


Figure 4.15: Terminal Settings

Now that the Zynq PL is programmed, and the Terminal connection has been created, we can program the Zynq PS with our software application.

(bn) In Project Explorer, right-click on **LED_Controller_test** and select ***Run As > Launch on Hardware (GDB)***, as shown in Figure 4.16.

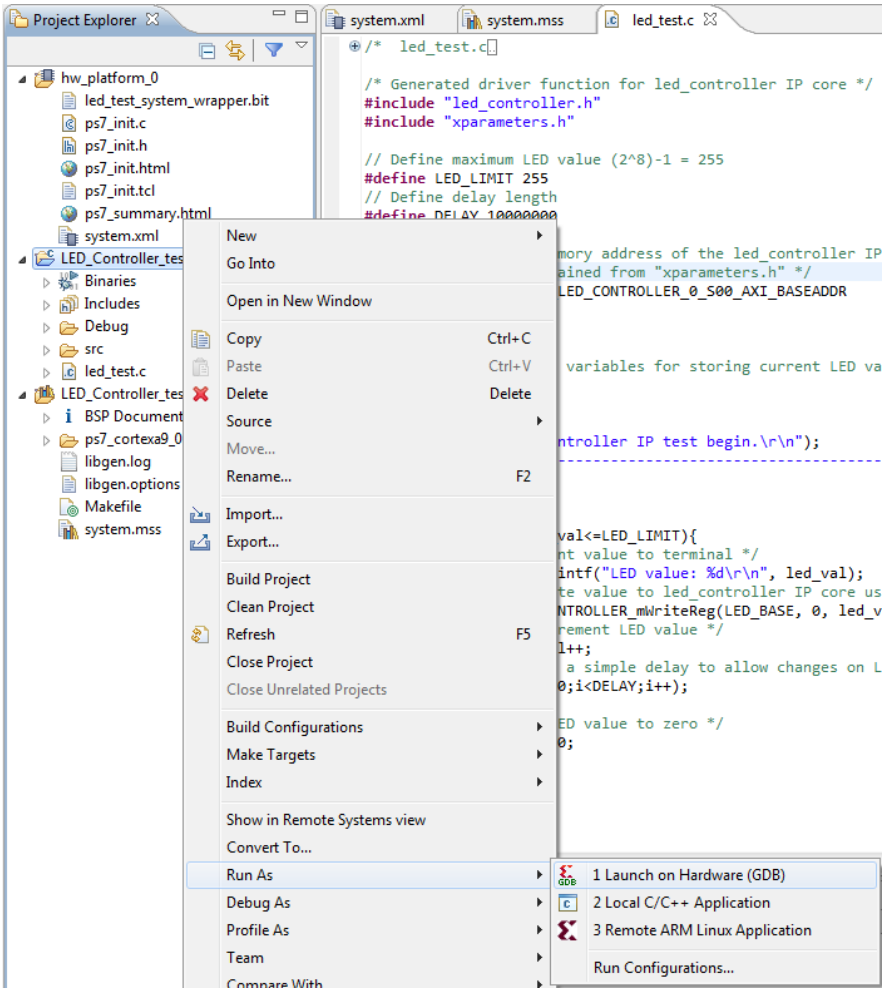


Figure 4.16: Run Application on hardware

(bo)Switch to the Terminal tab of the Console window, and confirm that the LED value is being output, as in Figure 4.17.

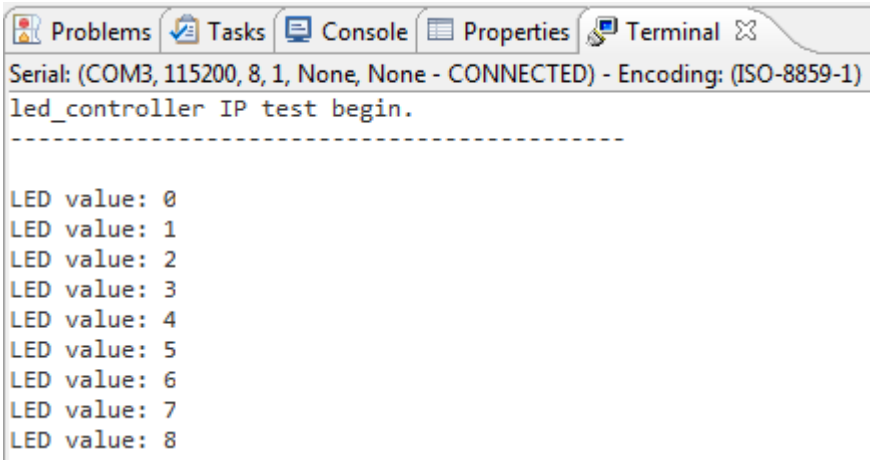


Figure 4.17: Terminal tab displaying LED values

You should also see the LEDs on the ZedBoard displaying the corresponding LED values.

This concludes this exercise on designing Zynq IP in HDL. You should now be familiar with:

- Creating AXI interface templates with the Create and Package IP Wizard.
- Adding functionality to HDL IP peripherals in Vivado and IP Packager.
- How to connect packaged IP to a Zynq Processing System in IP Integrator.
- Creating software applications to control the HDL IP using the generated C software drivers, and executing them on the ZedBoard.

Exercise 4B Creating IP in MathWorks HDL Coder

In this exercise, we will be creating an IP core which will perform the function of an LMS noise cancellation filter. Mathworks HDL Coder will be used to transform an existing Simulink block-based model into an RTL description which will be packaged for use in the Vivado IP Catalog. The...

We will start by opening the Simulink model in MatLab.

Before starting this exercise, you are required to copy some source files into a new working directory.

(a) In Windows Explorer, navigate to **C:\Zynq_Book\sources\hdl_coder_lms** and copy the contents of the directory to a new directory called **C:\Zynq_Book\hdl_coder_lms**.

(b) Launch MatLab by navigating to **Start > All Programs > MATLAB > R2013a > MATLAB R2013a**

Note: This workbook uses version R2013a of MatLab. If you have a different MatLab version you will need to replace R2013a with your own version (i.e. R2012a/R2012b).

MatLab will open and you will see the main workspace, as shown in Figure 4.18(or a variation thereof).

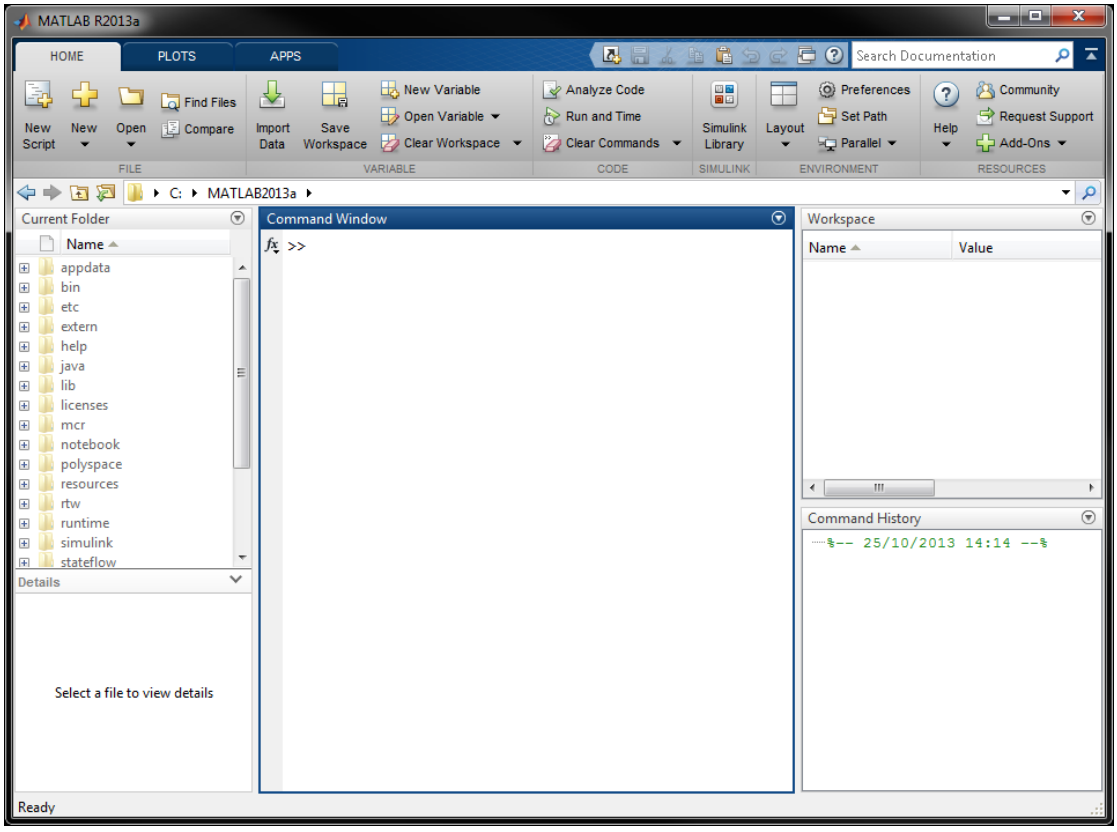


Figure 4.18: MatLab workspace environment

(c) Enter **C:\Zynq_Book\hdl_coder_lms** as the working directory, as highlighted in Figure 4.19.

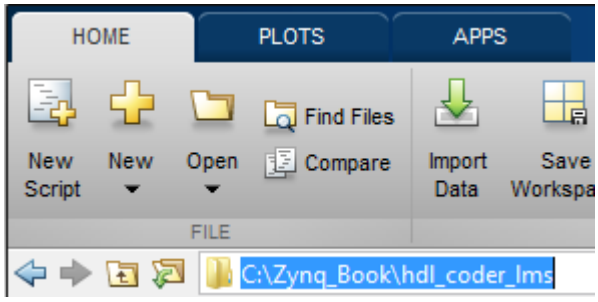


Figure 4.19: Setting the MatLab working directory

In the *Current Folder* pane, you should also see four files:

- **original_speech.wav** — A short audio clip of speech.
- **setup.m** — Performs setup commands to import the audio samples into the MatLab workspace and set the system sample rate accordingly.
- **lms.slx** — A simulink model which implements and LMS noise cancellation process.
- **playback.m** —Can be used to verify the LMS filtering process via audio playback of the various stages.

The setup commands in setup.m are automatically called when the Simulink simulation is initialised.

(d) Open the LMS Simulink model by double-clicking on ***lms.slx*** in *Current Folder* pane.
The model should open and you should see the LMS system, as shown in Figure 4.20.

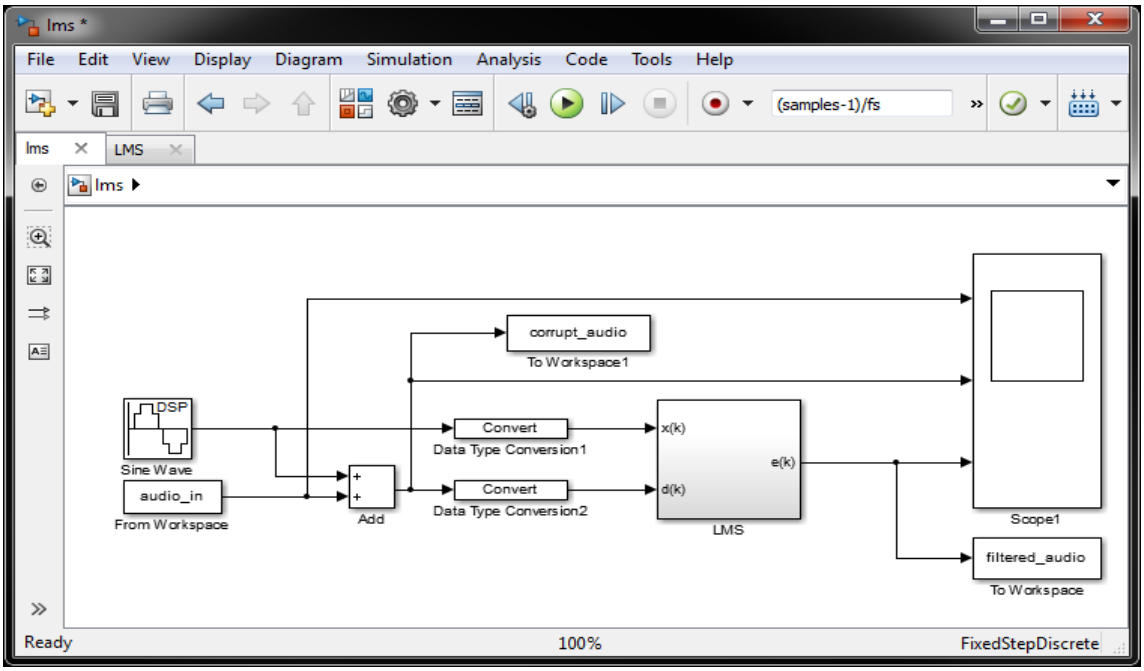


Figure 4.20: LMS model in Simulink

The model features two sources:

- a ***Sine Wave*** block which generates tonal noise.
- A ***From Workspace*** block which imports the audio samples from the MatLabe Workspace.

The tonal noise is then added to the audio samples to create a corrupted audio signal.

In order to generate HDL code for the Simulink LMS model using HDL Coder, the inputs to the system must be in fixed-point numerical format. Two ***Data Type Conversion*** blocks are used to convert the corrupt audio signal and the tonal noise signal to fixed-point format. The fixed-point signals are then input to an LMS subsystem, which we will explore in the next step.
At the output of the LMS subsystem, the error signal, **e(k)**, is input to a scope along with the corrupt audio and tonal noise inputs, for visual inspection of the signals. Two ***To Workspace*** blocks are also present to allow the LMS output and the corrupt audio signals to be output to the MatLab workspace for audio playback.

(e) Drill down into the LMS subsystem block by double-clicking on it. You will see the system in Figure 4.21.

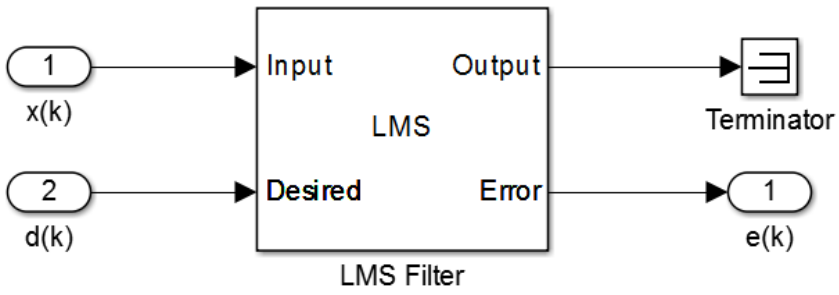


Figure 4.21: LMS subsystem

It features a single ***LMS Filter*** block. As we are not interested in the ***Output*** signal, it is unconnected.

(f) Open the *LMS Filter Block Parameters* by double-clicking on the ***LMS Filter*** block.
Take a moment to explore the parameters. You should be able to determine that there are 16 ***adaptive filter coefficients*** and a ***step size*** of 0.1.

(g) Close the Parameters window, and return to the main Simulink model by clicking the ***Up To Parent*** button,  .

We will be generating HDL code for the LMS subsystem only.

Right-click on the LMS subsystem and select **HDL Code > HDL Workflow Advisor**.

The HDL Workflow Advisor window will open, as in Figure 4.22.

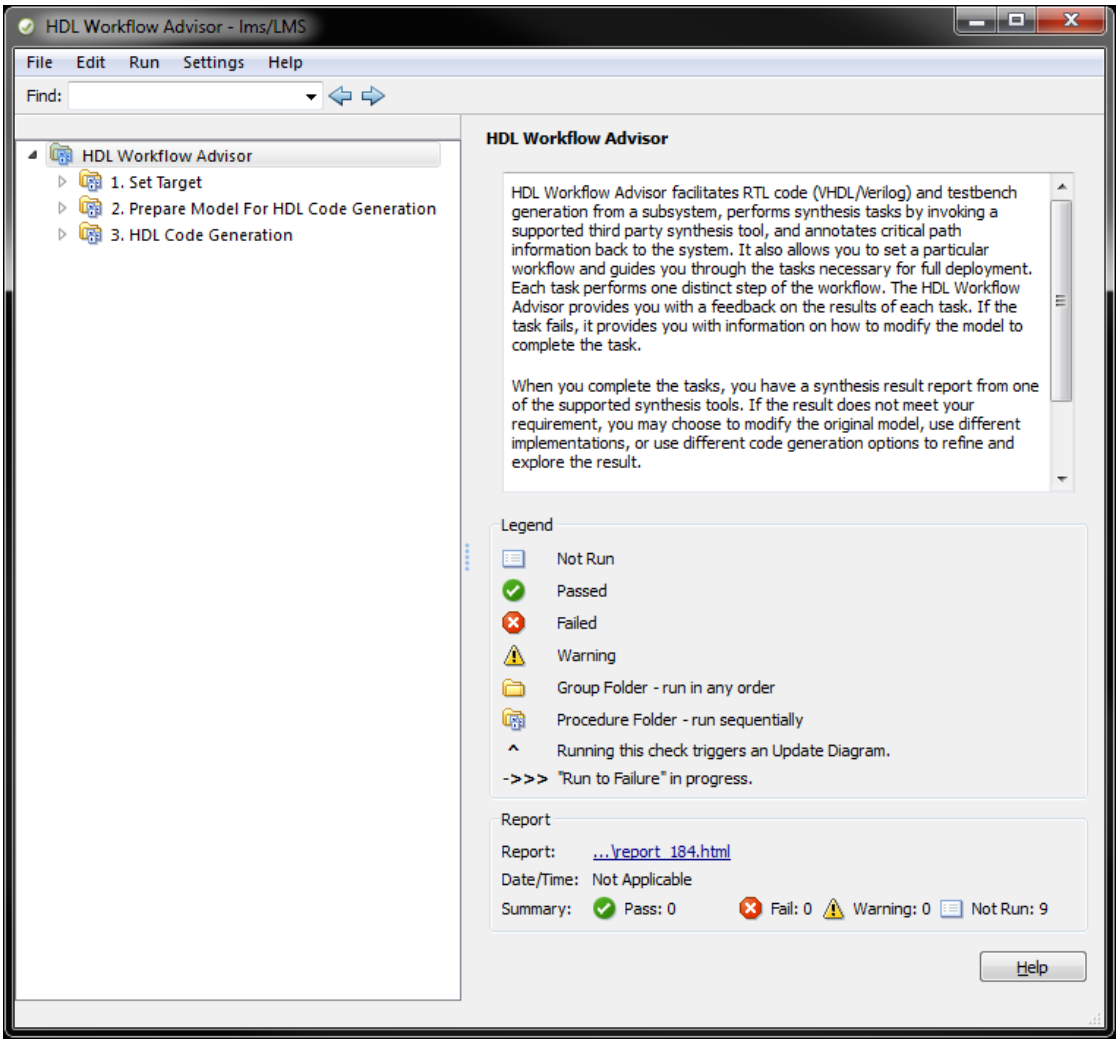


Figure 4.22: HDL Workflow Advisor window

The HDL Workflow Advisor guides you through the steps required to generate RTL code for your design.

(h) In the left-hand panel, expand ***Set Target*** and select ***1.1. Set Target Device and Synthesis Tool***.

Here we specify the output format of the RTL and the target platform.

(i) In the *Input Parameters* pane, select ***IP Core Generation*** as the *Target workflow*, and ***Generic Xilinx Platform*** as the *Target platform*, as shown in Figure 4.23.

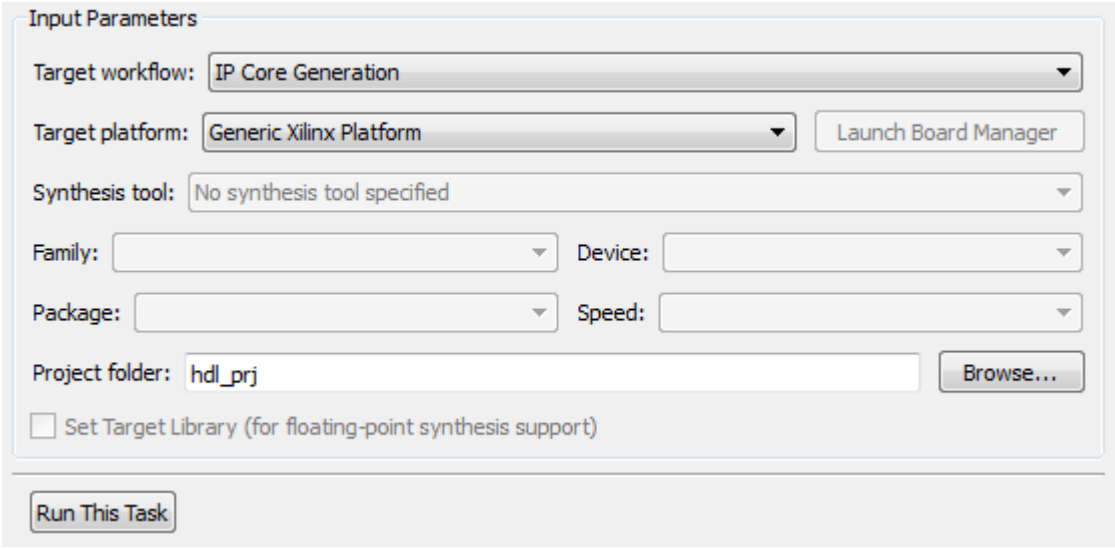


Figure 4.23: HDL Workflow Advisor Input Parameters

(j) Click ***Run This Task*** to apply the settings.

(k) Select ***Set Target Interface*** from the left hand panel.
Here we specify the target interface for the HDL code generation.
In the Input Parameters pane, select ***Coprocessing - blocking*** as the *Processor/FPGA synchronization*. This will automatically infer an **AXI4-Lite** interface for all ports in the design, and specify a memory address for each.

(l) Click ***Run This Task*** to apply the settings.

(m) Expand ***Prepare Model for HDL Code Generation*** in the left hand panel, and select ***Check Global Settings***.

Here, model-level settings will be checked to verify if the model is ready for HDL code generation.

(n) Click ***Run This Task*** to check the model-level settings.

If this step fails, click ***Modify All*** to allow *HDL Workflow Advisor* to modify the settings.
This step should now pass, and you will be presented with a table of the results.

The next few steps are all checks, and can be performed in batch.

(o) Right-click on *Check Sample Times* in the left hand pane, and select ***Run to Selected Task***.
This will perform the checks one after another to prevent you from running each individually.
All check should pass.

The final steps involve specifying basic settings about the RTL code, such as what language to use (VHDL/Verilog), and what code generation reports to generate. Finally the HDL code will be generated.

(p) Expand *HDL Code Generation* in the left hand pane, and further expand ***Set Code Generation Options***.
Click on ***Set Basic Options***.

(q) Select ***VHDL*** as the **Language** in the *Target* pane.
You can also select any of the *Code generation reports* that you would like.

(r) Select ***Set Advanced Options*** in the left hand panel.
Here you can specify more advanced options for the HDL code.
We will be leaving the values as default, but you may wish to explore the settings for future use.

(s) Right-click on *Set Advanced Options*, and select ***Run to Selected Task*** to apply the settings.

(t) Finally, select *Generate RTL Code and IP Core* from the left hand panel.
This is the step which will finally generate the HDL code for out LMS IP Core.
Set the *IP core name* as ***lms_pcore*** and click ***Run This Task***.

Once HDL Coder has finished generating the HDL code, the Code Generation Report window will open. This provides a summary of the HDL Coder results and provides further information on the target interface and clocking.

The final stage of creating our LMS IP core is to package it with IP Packager so that we can use it in IP Integrator designs. To do this we will need to create a new Vivado project.

(u) Launch Vivado and create a new project called **lms_packaging** at the following location: **C:\Zynq_Book\hdl_coder_lms**, ensuring that the option to ***create a project subdirectory*** is selected. Also select the ***VHDL*** as the *target language*, and the ***ZedBoard*** as the *default part*.

For more detail on the process of creating a new Vivado project, refer to **Step (a)** of **Exercise 4A**.

(v) When the project has been created and opened, select ***Tools > Create and Package IP*** from the *menu bar*, and Click ***Next***.

(w) Select the option to ***Package a specified directory***, and click ***Next***.

(x) Enter **C:/Zynq_Book/hdl_coder_lms/hdl_prj/ipcore/lms_pcore_v1_00_a** as the *IP Location*.

(y) Click ***Next*** to move to the *Edit in IP Packager Project Name* dialogue, and click Next to accept the default *Project Name* and *Project Location*.

(z) At the *Summary* window, and click ***Finish*** to launch **IP Packager**.

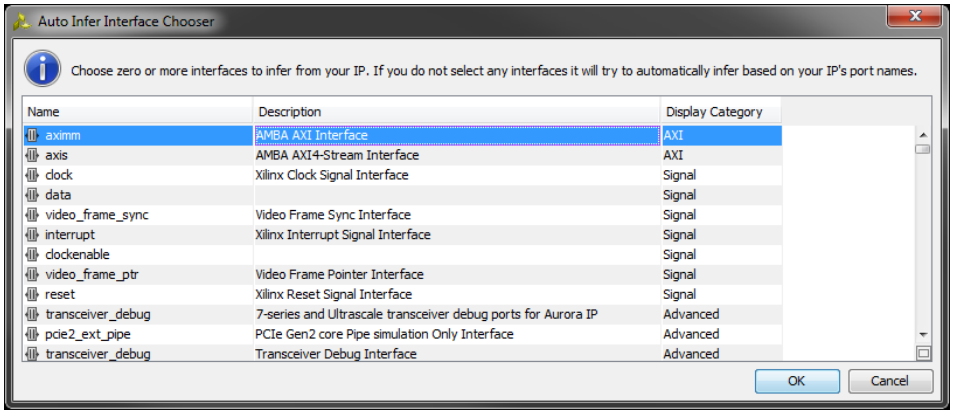
(aa) In the left hand panel of the *IP Packager* window, select ***IP Ports and Interfaces***.

The *IP Interfaces* panel will open, and you should see that **IP Packager** has identified the individual AXI ports, but has not inferred an AXI interface.

To infer an AXI interface:

(ab) Right-click on a blank section of the *IP Ports and Interfaces* pane, and select ***Auto Infer Interface ...***

(ac) The *Auto Infer Interface Chooser* window will open:



Select ***aximm*** from the list, as shown, and click ***OK***.

The individual AXI ports in our design will be mapped to an **AXILite** interface.

(ad) Select ***IP Addressing and Memory*** from the left hand panel. Here, IP Packager has incorrectly specified an address **Range** of **4294967296**. Click on the **Range**, and change the value to ***32***.

(ae) Finally, select ***Review and Package*** from the left hand menu.

Review the information provided, and click ***Package IP***.

This completes the generation of an LMS component from Mathworks HDL Coder. You should now be familiar with:

- Using the Simulink block-based design environment for the design and simulation of IP.
- Using the HDL Workflow Advisor to guide you through the steps of generating RTL code

and IP cores for existing Simulink designs.

- Packaging HDL Coder generated IP blocks in IP Packager for use in Vivado IP Integrator designs.

Exercise 4C Creating IP in Vivado HLS

In this final exercise, we will creating an IP core that will implement the functionality of an NCO. The tool that we will be using is Vivado HLS, and we shall explore some of the features which allow us to specify arbitrary precision fixed-point data types, as well as the directives required to export IP with an AXI-Lite slave interface, to allow the IP core to interface with the Zynq processor.

We will start by creating a new project in Vivado HLS.

(a) Launch Vivado HLS by double-clicking on the Vivado HLS desktop icon:  , or by navigating to **Start > All Programs > Xilinx Design Tools > Vivado 2014.1 > Vivado HLS > Vivado HLS 2014.1**

(b) When Vivado HLS loads, you will be presented with the *Getting Started* screen, as in Figure 4.24.

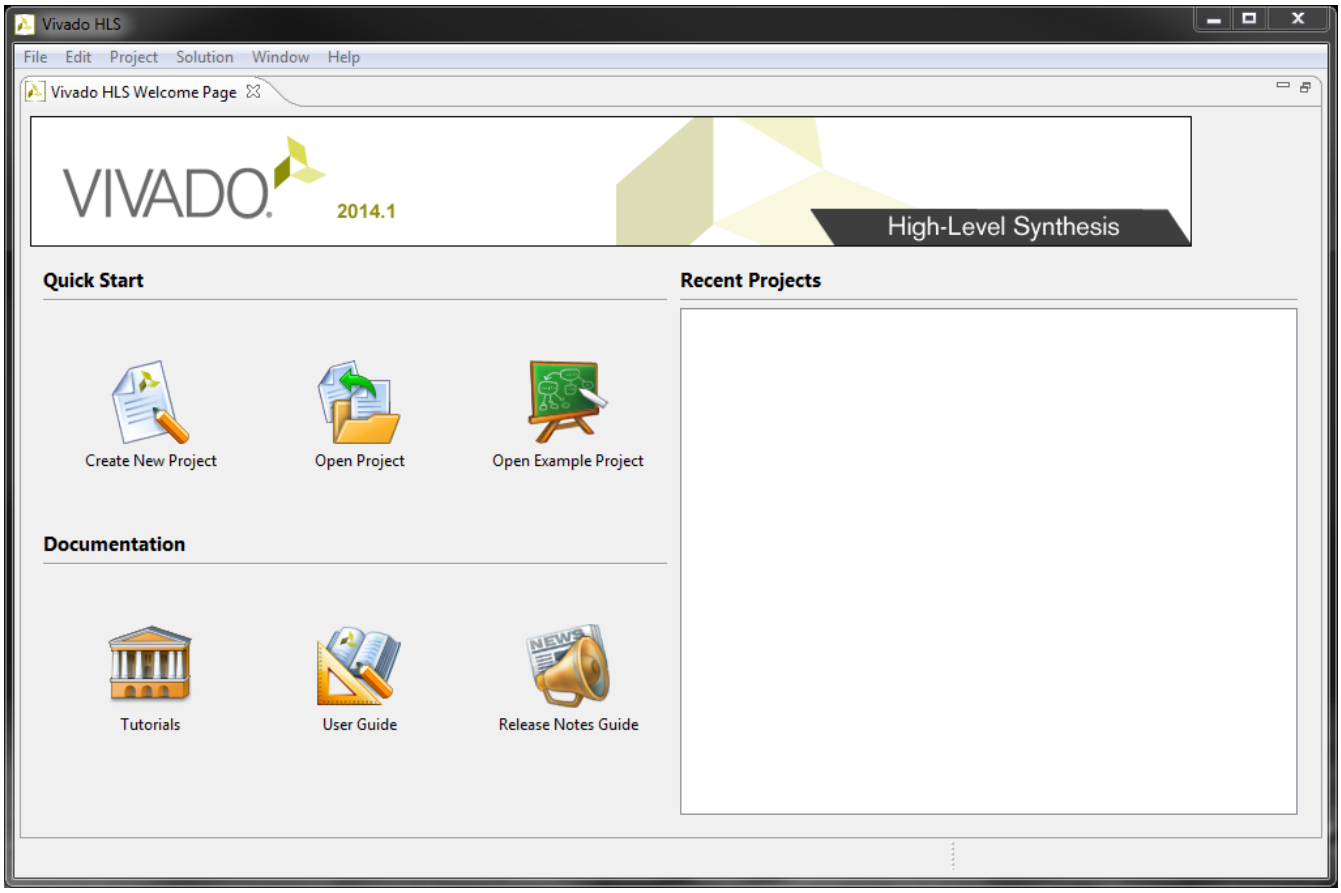


Figure 4.24: Vivado HLS Getting Started screen

(c) Select the option to ***Create New Project*** and the *New Vivado HLS Project Wizard* will open, as in Figure 4.25.

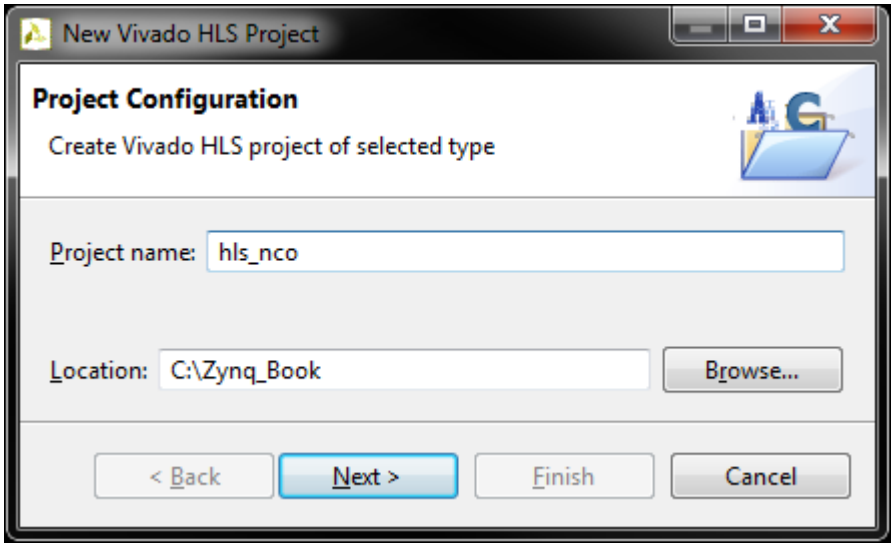


Figure 4.25: Vivado HLS New Project Wizard

Enter ***hls_nco*** as the *Project name*, and ***C:\Zynq_Book*** as *Location*.

Ensure that the options match those in Figure 4.25, and click ***Next***.

(d) The *Add/Remove Files* dialogue will appear. This is where existing C-based source files can be added to the project, or new files created.

Enter ***nco*** as the *Top Function* and click ***Add Files...***

Navigate to ***C:\Zynq_Book\sources\hls_nco*** and select ***nco.cpp***. Click ***Open***.

The dialogue should now resemble Figure 4.26.

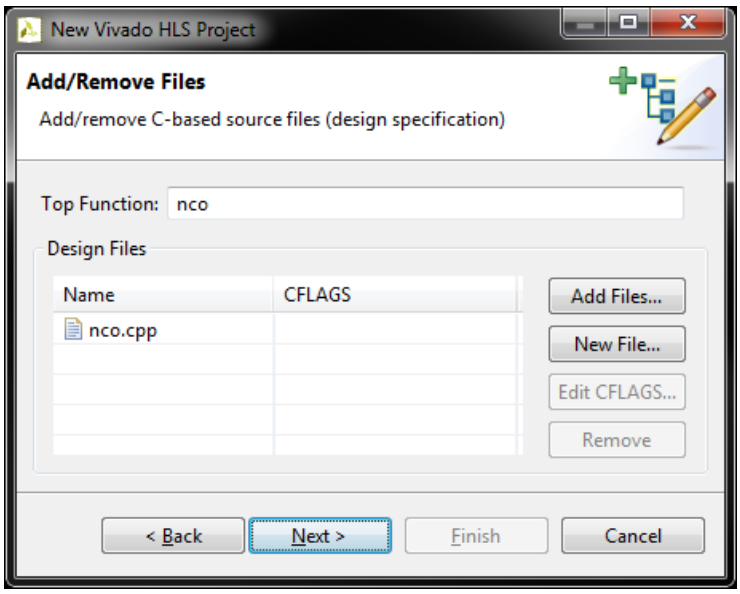


Figure 4.26: Vivado HLS New Project Wizard (Add/Remove Files)

Click ***Next***.

(e) A second *Add/Remove Files* dialogue will appear. This is where C-based testbench files can be added to the project, or new files created.

Click ***Add Files...*** and navigate to ***C:\Zynq_Book\sources\hls_nco***. Select ***nco_tb.cpp*** and click ***Open*** to add the testbench file to the project.

Click ***Next***.

(f) The *Solution Configuration* dialogue will open. Here we will be selecting the part which we will be targeting. In this particular case we will be targeting the Zynq-7020 on the ZedBoard, but if you have a different development board, it is easy to choose your particular board instead.

Click the selection button, ..., in the *Part Selection* pane.

The *Device Selection Dialog* will open.

As we are targeting the ZedBoard, select ***Boards*** in the *Specify* pane and choose ***ZedBoard Zynq Evaluation and Development Kit***, as in Figure 4.27.

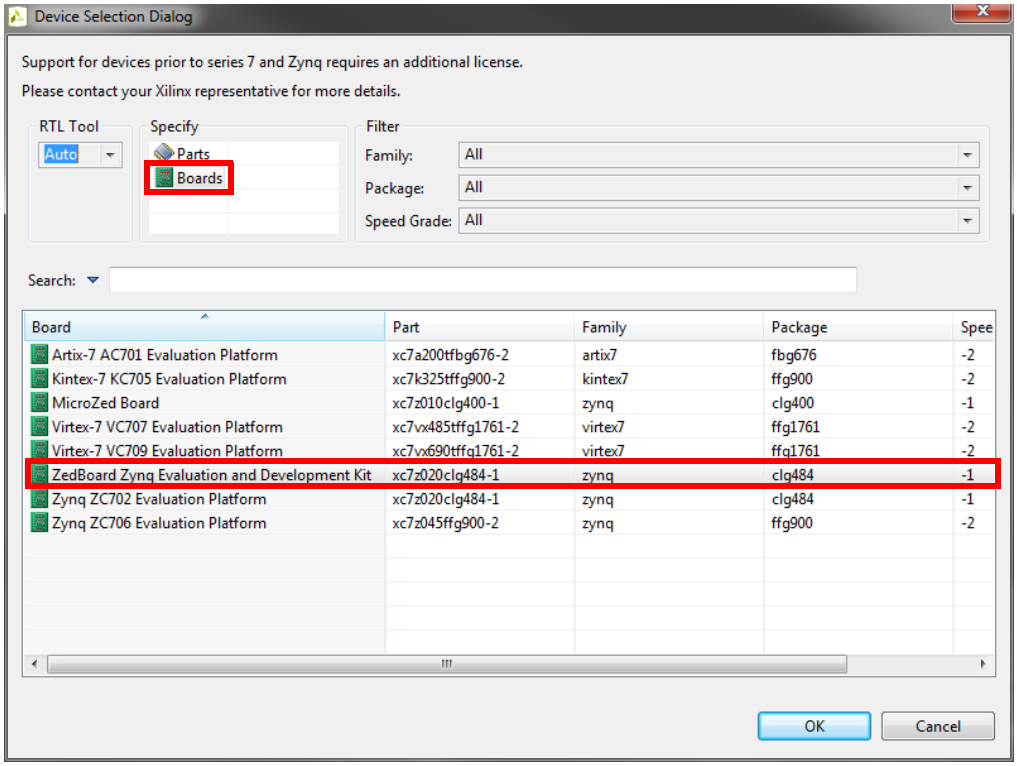


Figure 4.27: Vivado HLS Device Selection Dialog

Click ***OK*** to close the dialogue and return to the *New Project Wizard*.

(g) Click ***Finish*** to close the *New Project Wizard* and to create the project.

The Vivado HLS workspace will open.

(h) In the *Explorer* panel, expand the ***Source*** and ***Test Bench*** headings. You should see the source files that we specified in the *New Project Wizard*, as in Figure 4.28.

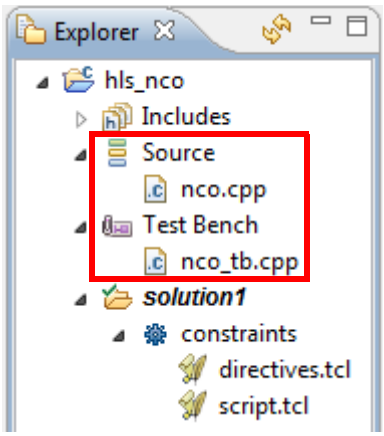


Figure 4.28: Vivado HLS Explorer panel

(i) Open ***nco.cpp*** and examine the contents of the file.

You should notice the inclusion of the header file ***ap_fixed.h*** on the first line. This is the arbitrary precision fixed-point library which adds support for the use of fixed-point data types in C++.

The next thing that you should see is the global declaration of a 2^{12} = 4096 value array:

```
const ap_fixed<16,2> sine_lut[4096] ...
```

This forms the sinewave lookup table. It is defined as an array of type `ap_fixed<16,2>`, which means that all values are16-bit, signed fixed-point (2 integer bits and 14 fractions bits). Further information on fixed-point data types in Vivado HLS can be found in ***Chapter15 - Vivado HLS: A Closer Look*** of the **Zynq Book.**

The functionality of the NCO is contained in the function:

```
void nco (ap_fixed<16,2> *sine_sample, ap_ufixed<16,12> step_size)
```

It takes two arguments:

- **`*sine_sample`** — A pointer to a 16-bit, signed fixed-point variable which forms the output sample of the NCO.
- **`step_size`** — 16-bit, unsigned fixed-point value which provides the step size input for the NCO.

(j) Explore the **`nco`** function, ensuring that you understand it all.

Open ***nco_tb.cpp***. This is the testbench file which is used to ensure that the functionality of the C-based source file is correct.

Explore the code in the file, ensuring that you understand the functionality.

This is a simple file which opens a text file in write-mode, to allow you to output the sinusoidal samples. It then calls the **`nco`** function from within a *for-loop* in order to generate a finite

number of sinusoidal samples, which are then output to the text file.

The text file is formatted in a way which easily allows you to import the samples into MatLab for analysis.

Note: The location of the output file is determined by the following line in the testbench file:

```
char *outfile = "E:\\nco_sine.m";
```

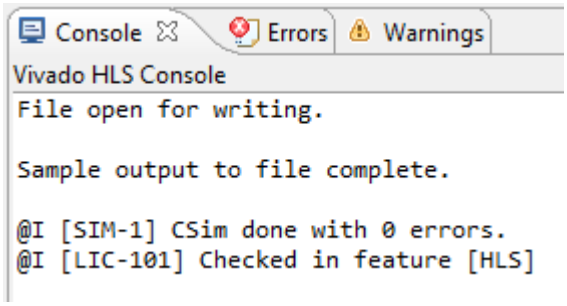
You should change the output file path accordingly to a location on your local machine.

We will now run a C simulation.

(k) Click the ***Run C Simulation*** button, , from the *Main Toolbar*.

The *C Simulation Dialog* window will open. Click ***OK*** to run the simulation with the default settings.

The C simulation will run, and you should see the following output in the Console window:



The sine wave samples that were generated by the NCO will have been output to the location which you specified in the previous step.

If you wish, you can import the sine wave samples into MATLAB using the output file to verify that the NCO has correctly generated a sine wave. This should be done at your own discretion, and will not be covered in this exercise.

The process of HLS has been covered previously in **The Zynq Book Tutorial: Designing With Vivado High Level Synthesis**, and you should refer to it for more detailed information on the various steps involved. For the purposes of this exercise, it is presumed that you have a reasonable knowledge of the Vivado HLS tool.

As we want to allow our NCO peripheral to be controlled by a Zynq PS, it is necessary to give it an AXI interface. This is done in Vivado HLS through the use of directives.

(l) Ensure that **nco.cpp** is the active source file, and select the ***Directive*** tab in the right-hand side of the Vivado HLS workspace, as shown in Figure 4.29.

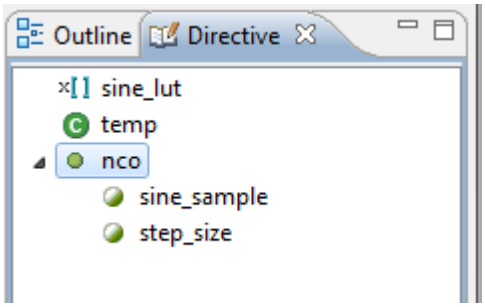


Figure 4.29: Vivado HLS Directive tab

First, we will define the interface of the NCO as an **AXI-Lite slave**.

(m) Right-click on ***nco*** in the *Directive* tab, and select ***Insert Directive***.
As the *Directive Type*, select ***RESOURCE***.
and select ***AXI4LiteS [adapter]*** as the *core*, from the pop-up list.
Leave *Destination* as ***Directive File***, and click ***OK***.

We will now define the NCO as having a **ap_ctrl_none** interface, to remove unneeded control signals.

(n) Right-click on ***nco*** in the *Directive* tab, and select ***Insert Directive***.
As the *Directive Type*, select ***INTERFACE***.
and select ***ap_ctrl_none*** as the *mode*, from the drop-down list.
Leave *Destination* as ***Directive File***, and click ***OK***.

Finally, we will be defining the two variables, **sine_sample** and **step_size**, as ports on the **AXI-LIte slave** interface.

(o) Right-click on ***sine_sample*** in the *Directive* tab, and select ***Insert Directive***.
As the *Directive Type*, select ***RESOURCE***.
and select ***AXI4LiteS [adapter]*** as the *core*, from the pop-up list.
Leave *Destination* as ***Directive File***, and click ***OK***.

(p) Repeat the previous step for the **step_size** variable in the *Directive* tab.

On completion, the Directive tab should look like Figure 4.30.

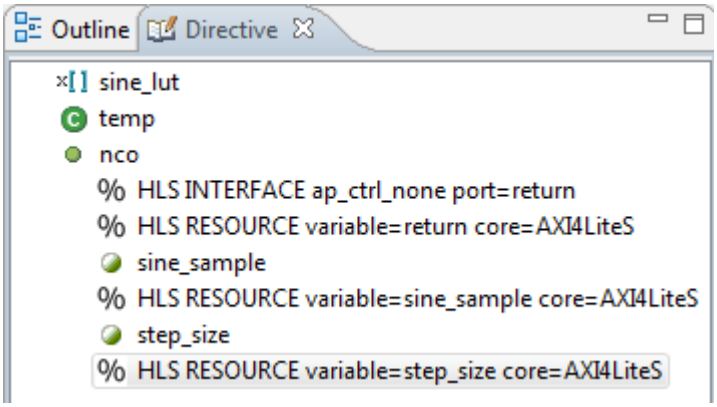


Figure 4.30: Complete Directive tab

We can now run HLS.

(q) Run C Synthesis by clicking the ***Run C Synthesis*** button, , from the *Main Toolbar*.

(r) Click the Export RTL button, , from the Main Toolbar.

The Export RTL Dialog window will open, as shown in Figure 4.31.

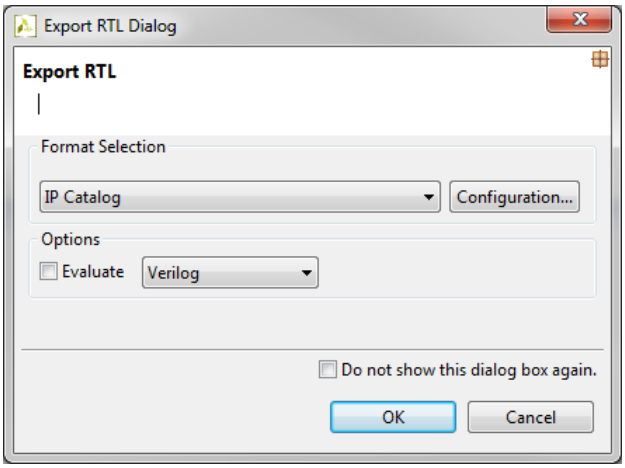


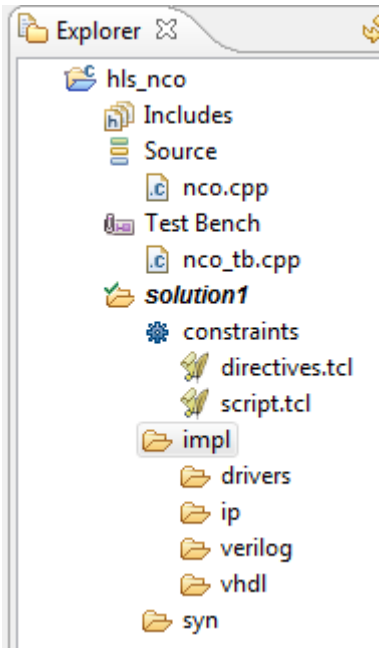
Figure 4.31: Vivado HLS Export RTL Dialog Window

(s) Select IP Catalog as the Format Selection.

If you choose, you can edit the *IP Identification* data by clicking the ***Configuration*** button.

(t) Click ***OK*** to generate the IP core.

When RTL Generation has completed, a directory named ***impl*** will be visible in the *Explorer* panel



This directory contains the ***ip*** subdirectory which contains the generated IP package.

Take a moment to explore the contents of the ***ip*** directory.

With the IP generated, the next step would be to include it in an IP Integrator design (which will be covered in the next tutorial). For future reference, however, it is worth briefly describing how this would be done.

In order to include HLS generated IP in IP Integrator, it must first be added to the Vivado IP Catalog. To do this you must add the output from HLS to an IP repository. This can be achieved by either adding the HLS generated output directory to an existing IUP repository directory, or by creating a new repository. In either case, the directory is the same. In this case:

C:\Zynq_Book\hls_nco\solution1\impl\ip

We have now completed the generation of the NCO component as an IP Integrator compatible AXI-Lite block. You should now be familiar with:

- Specifying directives in Vivado HLS designs which define the control interface of the exported RTL.
- The process of specifying and AXI4 interface for a design, to enable a Vivado HLS system to be easily connected to the Zynq PS.
- Exporting a Vivado HLS design as an IP core that is compatible with the Vivado IP Catalog and IP Integrator.